

Software Requirements Specification (SRS)

Campus Classroom & Lab Resource Booking System

Document version	1.0 (Draft for team review)
Date	30 July 2026
Status	Baseline candidate — pending sign-off on Open Issues (§9)
Standard followed	IEEE 830-style structure

1. Introduction

1.1 Purpose

This document specifies the functional and non-functional requirements for a Resource Booking System (RBS) that manages the reservation of a fixed set of institutional classrooms and computer labs. It is the controlling requirements baseline for all downstream deliverables (ER diagram, database schema, integration plan, test plan).

1.2 Scope

The RBS allows authorised campus users to browse, request, approve and audit bookings for a fixed, closed inventory of 11 physical rooms (8 labs, 3 classrooms). The system enforces non-overlapping allocation of a room in a given time slot, resolves competing simultaneous requests deterministically, applies a user-priority hierarchy, and produces descriptive utilisation and booking-count reports.

In scope

- Role-based access control for three user classes
- Resource browsing with live availability for a chosen date/time
- Booking request → validation → approval workflow
- Conflict detection and prevention of double-booking
- Concurrency control for simultaneous requests on the same resource (mutual exclusion)
- Booking history (per user) and audit logs (per resource, system-wide)
- Calendar view of resource occupancy
- Priority-based resolution between user classes
- Descriptive reports supporting the project KPIs (utilisation %, booking counts)

Out of scope

- Predictive analytics — all reports are descriptive/historical only. No demand forecasting, no recommendation engine.
- Hardware integration — no smart locks, door access controllers, RFID, biometric readers, occupancy sensors, or projector/AV control.
- Payment or fee collection
- Native mobile applications
- Changes to the room inventory itself beyond Admin add/remove of records in the existing set (see §3.6 constraint DC-1)

1.3 Definitions, Acronyms and Abbreviations

Term	Definition
Resource	A bookable physical room — a lab or a classroom
Slot	A contiguous, bounded time interval on a single calendar date for which a resource may be reserved
Booking	A confirmed (APPROVED) reservation of one resource for one slot
Request	A submitted, not-yet-decided reservation attempt (status PENDING)
Conflict	Two or more requests/bookings whose slots overlap on the same resource

Term	Definition
Overlap	For slots A and B: $A.start < B.end$ AND $B.start < A.end$ (half-open interval [start, end))
Mutex	Mutual-exclusion lock guaranteeing that only one booking transaction evaluates a given resource + date at a time
Race condition	Two concurrent transactions each reading "available" before either writes, producing a double-booking
Utilisation	$\text{Booked hours} \div \text{available hours}$ for a resource over a period, expressed as a percentage
TAT	Turnaround time — interval between request submission and approval/denial
RBAC	Role-Based Access Control
Preemption	Displacement of a lower-priority approved booking by a higher-priority request

1.4 References

- Handwritten resource inventory register (photograph, 30 July 2026) — source of truth for §3.6 resource master data
- Team "Common Responsibilities" deliverables list (SRS, ER diagram, DB schema, coding standards, folder structure, naming conventions, API interfaces, Git structure, UI standards, integration plan, test plan)

1.5 Overview

Section 2 gives the overall product description and user classes. Section 3 states the numbered functional requirements and the fixed data requirements. Section 4 states non-functional requirements. Section 5 defines external interfaces. Section 6 defines the state model. Section 7 defines reporting and KPIs. Section 8 gives requirement traceability. Section 9 lists open issues requiring a decision before schema freeze.

2. Overall Description

2.1 Product Perspective

The RBS is a new, self-contained, single-institution web application. It replaces a manual/register-based booking process. It does not integrate with any existing campus ERP, LMS, or timetable system in this release; the academic timetable, if required as a baseline occupancy, must be loaded as ordinary bookings by an Admin (see Open Issue OI-3).

2.2 Product Functions (summary)

1. Authenticate a user and resolve their role
2. List resources with availability for a selected date and time window
3. Accept a booking request and reject overlaps at submission time
4. Route requests to approvers and record the decision
5. Serialise concurrent requests on the same resource so that at most one succeeds
6. Apply the priority hierarchy when requests compete
7. Present a calendar view of occupancy
8. Maintain per-user booking history
9. Maintain an immutable, system-wide audit log

10. Generate descriptive utilisation and booking-count reports

2.3 User Classes and Characteristics

ID	User class	Characteristics	Permissions	Priority
U1	Faculty	Teaching staff booking rooms for academic sessions, remedial classes, exams	Request; view own history	1 (highest)
U2	Lab Assistant	Technical staff booking labs for maintenance, imaging, hardware servicing	Request maintenance blocks on labs; mark a lab UNAVAILABLE; view lab history	2
U3	Admin	System owner / office staff	Full CRUD on resources within the fixed inventory; user & role management; override/cancel any booking; access all reports and audit logs	Override authority

Technical expertise assumed: low to moderate. Interface must be operable without training.

2.4 Operating Environment

Client	Modern desktop/mobile web browser (Chrome, Firefox, Edge, Safari — current and one prior major version)
Server	Application server hosted on campus infrastructure or a single cloud VM
Database	A single relational DBMS instance supporting ACID transactions and row-level locking (mandatory — see NFR-C1)
Network	Campus LAN/Wi-Fi; internet access not assumed for internal use

2.5 Design and Implementation Constraints

ID	Constraint
DC-1	Fixed inventory: The resource table shall contain only the rooms enumerated in §3.6. The system shall not permit creation of resources outside this set. Admin "add/remove" is limited to activating/deactivating and editing attributes of these records.
DC-2	Mutual exclusion mandatory: Booking confirmation shall be protected by an explicit mutual-exclusion mechanism. Optimistic "check-then-insert" without locking or a uniqueness guarantee is non-compliant.
DC-3	Descriptive only: No requirement shall be satisfied by a predictive or ML-based component.
DC-4	Audit log records are append-only; no interface shall expose update or delete on them.
DC-5	All timestamps stored in a single timezone (IST, UTC+05:30) or as UTC with consistent conversion at the presentation layer. The choice must be recorded in the coding standards document.

2.6 Assumptions and Dependencies

ID	Assumption
A-1	User accounts and role assignments are provisioned by an Admin; there is no public self-registration.
A-2	The bookable day is bounded (see Open Issue OI-2) — the system does not offer 24-hour booking by default.
A-3	Room capacity values are not present in the source register and will be supplied by the institution (see Open Issue OI-1).
A-4	A single institution, single campus, single academic calendar.

3. Specific Requirements — Functional

Priority key: M = Mandatory (must be in release 1), D = Desirable.

3.1 Authentication and Role-Based Access

ID	Requirement	Pri.
FR-1.1	The system shall authenticate every user by credentials before granting access to any function other than the login page.	M
FR-1.2	The system shall assign exactly one role from {Faculty, Lab Assistant, Admin} to each user account.	M
FR-1.3	The system shall enforce role permissions server-side on every request. Hiding a UI control shall not be the sole enforcement mechanism.	M
FR-1.4	The system shall deny and log any request for which the authenticated user's role lacks permission, returning an authorisation error without disclosing the target record's contents.	M
FR-1.5	Only Admin shall be able to create, deactivate, or change the role of a user account.	M
FR-1.6	The system shall terminate an idle session after a configurable timeout (default 30 minutes).	D

3.2 Resource Browsing and Calendar View

ID	Requirement	Pri.
FR-2.1	The system shall display a list of all active resources with room code, type (Lab/Classroom), block, and attributes.	M
FR-2.2	Given a date and a time window, the system shall display each resource's status as one of AVAILABLE, BOOKED, PENDING-REQUEST, or UNAVAILABLE (maintenance/blocked).	M
FR-2.3	The system shall allow filtering of the resource list by type, block, and availability.	M
FR-2.4	The system shall provide a calendar view presenting occupancy per resource in day and week granularity, with each block labelled by booked-by and purpose (subject to FR-2.6).	M

ID	Requirement	Pri.
FR-2.5	The calendar view shall visually distinguish approved bookings, pending requests, and maintenance blocks.	M
FR-2.6	Non-Admin users shall see the booking owner's name and purpose but shall not see any other user's contact details or internal remarks.	D
FR-2.7	Availability displayed to the user shall be advisory; final availability is determined only at commit time per FR-4.x. The UI shall not present availability as a guarantee.	M

3.3 Booking Request Workflow

ID	Requirement	Pri.
FR-3.1	An authenticated Faculty or Lab Assistant user shall be able to submit a booking request specifying resource, date, start time, end time, purpose, and expected headcount.	M
FR-3.2	The system shall reject a request where end_time <= start_time.	M
FR-3.3	The system shall reject a request whose date/time lies in the past.	M
FR-3.4	The system shall reject a request whose duration exceeds a configurable maximum (default 4 hours) or falls outside institutional operating hours.	M
FR-3.5	At submission, the system shall validate the requested slot against all existing APPROVED bookings for that resource and reject the request if an overlap exists, per the overlap rule in §1.3.	M
FR-3.6	A validated request shall be persisted with status PENDING and shall be routed to the appropriate approver per §3.5.	M
FR-3.7	The requester shall be able to cancel their own PENDING request, and their own APPROVED booking up to a configurable cut-off before start time (default 2 hours).	M
FR-3.8	Cancellation of an APPROVED booking shall immediately release the slot for booking by others.	M
FR-3.9	The system shall prevent the same user from holding two APPROVED bookings whose slots overlap, across different resources.	D
FR-3.10	The system shall support a recurring request (e.g. weekly for N weeks), expanded into individual bookings, each independently validated; the requester shall be shown which instances failed validation.	D

3.4 Conflict Detection and Concurrency Control

ID	Requirement	Pri.
FR-4.1	The system shall guarantee that no two APPROVED bookings for the same resource have overlapping slots, under any sequence or timing of concurrent requests.	M
FR-4.2	Booking confirmation shall execute inside a single database transaction that acquires a mutual-exclusion lock keyed on (resource_id, booking_date) before the overlap check and holds it until commit or rollback.	M

ID	Requirement	Pri.
FR-4.3	The overlap check and the write shall be atomic — no code path shall permit a read of availability, a release of the lock, and a later write based on the earlier read.	M
FR-4.4	Where two requests for an overlapping slot on the same resource are processed concurrently, exactly one shall succeed. The loser shall receive a distinct, user-comprehensible conflict message naming the conflicting slot, not a generic system error.	M
FR-4.5	The system shall additionally enforce non-overlap by a database-level integrity guarantee (exclusion/unique constraint on resource + slot), so that a defect in application-layer locking cannot produce corrupt data.	M
FR-4.6	Lock acquisition shall use a bounded wait with timeout; on timeout the transaction shall roll back and the user shall be asked to retry. Indefinite blocking is non-compliant.	M
FR-4.7	Locks shall be acquired in a single, documented order to prevent deadlock; any deadlock detected shall be logged with both transaction identifiers.	M
FR-4.8	The system shall detect and surface soft conflicts — multiple PENDING requests competing for the same overlapping slot — to the approver as a single grouped decision, ranked by priority (§3.5) and then submission timestamp.	M
FR-4.9	Approving one request in a competing group shall automatically transition the remaining competing requests to DENIED with reason SLOT_TAKEN, within the same transaction.	M

3.5 Approval and Priority Resolution

ID	Requirement	Pri.
FR-5.1	The system shall implement a priority hierarchy: Faculty (1) > Lab Assistant (2), with Admin holding override authority over all.	M
FR-5.2	Faculty requests shall be auto-approved if no conflict exists at submission; Lab Assistant requests shall require explicit approval.	M
FR-5.3	The approver may override the recommended ordering; an override shall require a mandatory textual justification, which shall be written to the audit log.	M
FR-5.4	Admin shall be able to cancel or reassign any APPROVED booking (preemption). Preemption shall require a reason and shall notify the displaced booking owner.	M
FR-5.5	Preemption shall be available to Admin only. A higher-priority user shall not automatically displace an existing APPROVED lower-priority booking; a first-come-first-served guarantee holds once approved.	M
FR-5.6	Every approval, denial, override and preemption shall record approver identity, timestamp, and reason.	M
FR-5.7	The system shall notify the requester of the decision (in-app notification; email desirable).	D
FR-5.8	PENDING requests unresolved by a configurable deadline (default 24 hours before slot start) shall be auto-denied with reason EXPIRED and the slot released.	D

3.6 Data Requirements — Resource Master (fixed inventory)

Per constraint DC-1, the resource table shall be seeded with exactly the following 11 records and no others.

Labs (8)

#	Room code	Type	Block	Notes
1	MB 409	Lab	MB	Machines available
2	MB 412	Lab	MB	Without machines
3	MB 413	Lab	MB	Machines available
4	MB 414	Lab	MB	Machines available
5	MB 407 A	Lab	MB	Machines available
6	MB 407 B	Lab	MB	Machines available
7	MB 408 A	Lab	MB	Machines available
8	MB 408 B	Lab	MB	Machines available

Classrooms (3)

#	Room code	Type	Block	Notes
9	AC 301	Classroom	AC	—
10	AC 304	Classroom	AC	—
11	AC 401	Classroom	AC	—

Notes on the source register:

- The entry in the classroom column beginning "MB 4..." is struck out in the source and is therefore excluded from the inventory.
- MB 412 is a lab without machines. It is modelled as type = Lab, has_machines = false — an attribute, not a separate resource type. Requests requiring machines shall be warned or blocked when targeting MB 412 (see FR-6.1).
- Capacity is absent from the source register and is required for the headcount validation in FR-3.1. See Open Issue OI-1.

ID	Requirement	Pri.
FR-6.1	The system shall record has_machines per lab and shall warn a requester who submits a machine-requiring purpose against a lab with has_machines = false.	D
FR-6.2	Admin shall be able to set a resource's status to ACTIVE or INACTIVE. An INACTIVE resource shall not appear as bookable but its historical bookings shall remain in reports and audit logs.	M
FR-6.3	The system shall reject any attempt — via UI or API — to insert a resource whose room code is not in the §3.6 list.	M

ID	Requirement	Pri.
FR-6.4	Deletion of a resource with existing bookings shall be blocked; deactivation shall be offered instead (referential integrity).	M

3.7 Booking History

ID	Requirement	Pri.
FR-7.1	Each user shall be able to view their own booking history: all past and future requests with resource, slot, status, approver and decision reason.	M
FR-7.2	History shall be filterable by date range, resource, and status, and sortable by date.	M
FR-7.3	Faculty shall be able to view the history of requests they have approved or denied.	M
FR-7.4	Admin shall be able to view the history of any user and any resource.	M
FR-7.5	Booking history shall be retained for a minimum of two academic years.	D

3.8 Audit Logs

ID	Requirement	Pri.
FR-8.1	The system shall maintain an append-only audit log capturing every state-changing event: request created, approved, denied, cancelled, preempted, resource activated/deactivated, user role changed, permission denied.	M
FR-8.2	Each audit record shall contain: event id, event type, actor user id, actor role, target entity type and id, previous state, new state, reason/justification, and server timestamp.	M
FR-8.3	The system shall provide a per-resource audit trail — the complete booking history of a single resource across all users — accessible to Admin.	M
FR-8.4	No interface, API endpoint, or application code path shall permit update or deletion of an audit record.	M
FR-8.5	The audit log shall be filterable by actor, resource, event type and date range, and exportable to CSV.	M
FR-8.6	Conflict events (FR-4.4 rejections and FR-4.9 auto-denials) shall be logged, as they are an input to the conflict-rate KPI.	M

4. Non-Functional Requirements

4.1 Performance

ID	Requirement
NFR-P1	Resource availability for a given date shall render within 2 seconds at the 95th percentile under normal load.

ID	Requirement
NFR-P2	A booking submission shall return a definitive outcome (confirmed / pending / conflict) within 3 seconds at the 95th percentile.
NFR-P3	The system shall support at least 50 concurrent active users and at least 20 simultaneous booking submissions without data corruption or unhandled error.
NFR-P4	A report over one academic term shall generate within 10 seconds.

4.2 Concurrency and Reliability

ID	Requirement
NFR-C1	The DBMS shall provide ACID transactions and row-level locking. Isolation level shall be at minimum REPEATABLE READ for booking transactions; the chosen level shall be documented.
NFR-C2	The system shall be verifiably free of double-booking under a concurrent load test issuing N simultaneous requests for one identical slot; the acceptance criterion is exactly one APPROVED booking and N-1 clean conflict responses.
NFR-C3	A failed booking transaction shall leave no partial state; rollback shall be complete.
NFR-C4	The system shall target 99% availability during institutional operating hours.
NFR-C5	The database shall be backed up daily with a documented restore procedure.

4.3 Security

ID	Requirement
NFR-S1	Passwords shall be stored only as salted cryptographic hashes; never in plaintext or reversible form.
NFR-S2	All traffic shall be served over HTTPS.
NFR-S3	The system shall be resistant to SQL injection (parameterised queries only), XSS (output encoding), and CSRF (token validation on state-changing requests).
NFR-S4	Authorisation shall be checked on every API call, including direct object references (no IDOR — a user shall not read or modify another user's booking by altering an id in the URL).
NFR-S5	Failed login attempts shall be rate-limited and logged.

4.4 Usability

ID	Requirement
NFR-U1	A first-time user shall be able to complete a booking request without written instructions.
NFR-U2	Error messages shall state the cause and the corrective action; conflict messages shall name the conflicting slot and offer the nearest available alternatives.
NFR-U3	The interface shall be responsive and usable at mobile viewport widths.

ID	Requirement
NFR-U4	Colour shall not be the sole carrier of status information (accessibility).

4.5 Maintainability and Portability

ID	Requirement
NFR-M1	The codebase shall conform to the team's agreed coding standards, folder structure and naming conventions documents.
NFR-M2	Business logic shall be separated from presentation, permitting the UI to be replaced without changing booking rules.
NFR-M3	Configurable parameters (operating hours, max duration, cancellation cut-off, session timeout, lock timeout) shall reside in configuration, not hardcoded.
NFR-M4	The application shall be deployable on Windows and Linux hosts.

5. External Interface Requirements

5.1 User Interfaces

- Login screen
- Dashboard — role-specific: pending actions, upcoming bookings
- Resource browser — list + filters + availability for a chosen date/time
- Calendar view — day/week occupancy grid per resource
- Booking request form
- Approval queue — grouped competing requests with priority ordering (Faculty/Admin)
- My history
- Admin console — resources, users, roles
- Audit log viewer with filters and CSV export
- Reports page

All screens shall follow the team's UI Design Standards document.

5.2 Software Interfaces

- Application ↔ RDBMS over a standard database driver
- Internal REST-style API; endpoints, request/response schemas and error codes to be specified in the API/Function Interfaces document derived from this SRS
- Optional SMTP interface for email notification (FR-5.7)

5.3 Hardware Interfaces

None. Explicitly out of scope.

5.4 Communication Interfaces

HTTPS over TCP/IP.

6. Booking State Model

From state	To state	Trigger
(none)	PENDING	submit (FR-3.1)
PENDING	APPROVED	approve (FR-5.2 / FR-5.3)
PENDING	DENIED	deny (FR-5.3)
PENDING	CANCELLED	cancel by owner (FR-3.7)
APPROVED	CANCELLED	cancel by owner before cut-off (FR-3.7)
APPROVED	PREEMPTED	Admin preemption (FR-5.4)
APPROVED	COMPLETED	end_time has passed

Terminal states: DENIED, CANCELLED, PREEMPTED, COMPLETED.

COMPLETED: APPROVED booking whose end_time has passed.

Rules:

- Only APPROVED and PENDING states block a slot from being newly requested; APPROVED is a hard block (FR-3.5), PENDING is a soft block (FR-4.8).
- Every transition writes an audit record (FR-8.1).
- Denial reasons shall be enumerated: SLOT_TAKEN, EXPIRED, INSUFFICIENT_PRIORITY, CAPACITY_EXCEEDED, MAINTENANCE, REJECTED_BY_APPROVER.

7. Reporting and KPIs

All reports are descriptive only (DC-3).

ID	Report
FR-9.1	Utilisation report — per resource, per period: booked hours, available hours, utilisation %. Aggregatable by type (Lab/Classroom) and block (MB/AC).
FR-9.2	Booking count report — count of requests by status, by user role, by resource, by period.
FR-9.3	Peak-load report — booking count by hour-of-day and day-of-week, identifying most and least contended slots. Historical description only; no forecasting.
FR-9.4	Conflict report — count of rejected-on-conflict submissions and auto-denials per resource per period.
FR-9.5	Approval TAT report — mean and median hours from submission to decision, by approver and by role.
FR-9.6	Cancellation / no-show report — cancellations by user and by lead time.

All reports shall be exportable to CSV and shall be filterable by date range, resource, type, block and role.

Project KPIs these reports serve

KPI	Definition	Traces to
Double-booking incidents	Count of overlapping APPROVED bookings	FR-4.1, NFR-C2 — target zero
Resource utilisation	Booked hrs ÷ available hrs	FR-9.1
Booking success rate	APPROVED ÷ total submitted	FR-9.2
Conflict rate	Conflict rejections ÷ total submitted	FR-9.4
Approval TAT	Median hours to decision	FR-9.5
Audit completeness	State changes with an audit record ÷ total state changes	FR-8.1 — target 100%

8. Requirements Traceability

Requirement area	Specified by
Role-based access	FR-1.1–1.6, NFR-S4
Resource browsing with status for a date/time	FR-2.1–2.3, FR-2.7
Booking workflow with overlap validation	FR-3.1–3.8, FR-5.2–5.3
Concurrency control / mutex / race conditions	FR-4.1–4.7, NFR-C1–C3, DC-2
Conflict detection	FR-3.5, FR-4.4, FR-4.8, FR-4.9, FR-9.4
Booking history	FR-7.1–7.5
Audit logs (entire resource history)	FR-8.1–8.6
DB must not exceed given classrooms and labs	§3.6, DC-1, FR-6.3
Utilisation and booking-count reports	FR-9.1, FR-9.2
Calendar view	FR-2.4, FR-2.5
User priority to be decided	FR-5.1, FR-5.4–5.7
Lab assistant maintenance	U2, FR-5.3, FR-6.2, denial reason MAINTENANCE
Descriptive not predictive	DC-3, §7
No hardware integration	§5.3, §1.2

9. Open Issues (decide before schema freeze)

ID	Issue	Impact	Recommended resolution
OI-1	Room capacities are not in the source register.	FR-3.1 headcount validation and the CAPACITY_EXCEEDED denial reason cannot be implemented.	Obtain actual figures; until then make capacity nullable and skip the check when null.
OI-2	Institutional operating hours and slot granularity (fixed 1-hour periods vs. free-form	Determines whether slots are discrete rows or interval ranges — a	Free-form intervals with 30-minute granularity, 08:00–18:00.

ID	Issue	Impact	Recommended resolution
	start/end times) are undefined.	fundamental schema decision affecting the overlap constraint.	
OI-3	Is the regular academic timetable a baseline occupancy the system must respect?	If yes, most slots are pre-occupied and the system is an exception booking tool, not a primary one. Large scope implication.	Admin bulk-loads timetable as recurring Faculty bookings at term start.
OI-4	Is preemption of an approved lower-priority booking by higher-priority academic need required?	FR-5.5 currently forbids it outside Admin action. If required, add notification and re-accommodation flows.	Admin-only preemption, as specified.
OI-5	Notification channel — in-app only, or email?	Adds an SMTP dependency and configuration.	In-app mandatory, email desirable.

Appendix A — Recommended entity set

(Input to the ER diagram deliverable.) Indicative only; the ER diagram and DB schema documents are the authoritative artefacts.

Entity	Purpose	Notes / attributes
USER	Accounts	FK to ROLE
ROLE	Faculty / Lab Assistant / Admin	Carries priority_rank
RESOURCE	The fixed 18 rooms	room_code UNIQUE; type, block, has_machines, capacity, status
BOOKING	Requests and bookings	resource_id, requested_by, booking_date, start_time, end_time, status, purpose, headcount, approved_by, decision_reason, created_at. Non-overlap constraint on (resource_id, slot) where status = APPROVED.
MAINTENANCE_BLOCK	Lab-assistant blocks	May be modelled as a BOOKING subtype rather than a separate entity
AUDIT_LOG	Append-only event trail	No update/delete path
NOTIFICATION	Decision notices	Optional per OI-5

Critical schema note: the non-overlap guarantee (FR-4.5) must be expressed as a database constraint, not only as application logic. In PostgreSQL this is an EXCLUDE USING gist constraint on a strange with a WHERE status = 'APPROVED' predicate. In a DBMS without range-exclusion support, the fallback is discrete fixed slots with a UNIQUE (resource_id, booking_date, slot_id) partial constraint — which forces the OI-2 decision toward fixed periods. This choice must be made deliberately, not discovered during coding.